Ingegneria del Software — Soluzione del Tema 07/02/2022

Esercizio 1

Si consideri la classe Java `JobsDB` per la gestione di domande e offerte di lavoro (classi `JobRequest` e `JobOffer`).

```
public class JobsDB {
  // Ritorna l'elenco delle richieste salvate.
  public /*@ pure @*/ Set<JobRequest> getRequests();

  // Ritorna l'elenco delle offerte salvate
  // in ordine cronologico dalla meno alla piu' recente.
  public /*@ pure @*/ List<JobOffer> getOffers();

  // Aggiunge una richiesta di lavoro.
  // Lancia una DuplicateException se la richiesta e' gia' presente.
  public void addRequest(JobRequest req) throws DuplicateException;

  // Aggiunge un'offerta di lavoro e ritorna l'insieme di tutte le richieste
  // di lavoro compatibili con l'offerta (senza rimuoverle)
  public Set<JobRequest> addOffer(JobOffer offer);

  // Ricerca un'offerta di lavoro compatibile con la richiesta req.
  // Se esiste almeno un'offerta di lavoro compatibile con la richiesta,
  // ritorna la prima offerta compatibile (meno recente) e la rimuove.
  // Altrimenti, ritorna null.
  public JobOffer getOffer(JobRequest req);
}

public /*@ pure @*/ class JobRequest {
  // Ritorna true se e solo se l'offerta e' compatibile con la richiesta.
  public boolean matches(JobOffer offer);
  ... }
```

Domanda a)

Si specifichi in JML il metodo `addOffer`.

Soluzione

Definiamo `unchangedRequests`, `unchangedOffers` come segue.

```
//@ unchangedRequests <==> (getRequests().size() == \old(getRequests()).size() &&
//@   getRequests().containsAll(\old(getRequests())
//@
//@ unchangedOffers <==> (getOffers().size() == \old(getOffers()).size() &&
//@   (\forall int i; i>=0 && i<getOffers().size();
//@      \old(getOffers()).get(i).equals(getOffers().get(i)) )
```

Possiamo ora definire `addOffer` come segue, ricordandosi di mantenere l'ordinamento cronologico di getOffers():

```
//@requires offer != null
//@
//@ensures unchangedRequests &&
//@ getOffers().size()==\old(getOffers().size())+1 &&
//@ (\forall int i; i>=0 && i<\old(getOffers()).size();
//@   getOffers().get(i).equals(\old(getOffers()).get(i)) ) &&
//@ getOffers().get(get(offers().size()-1).equals(offer) &&
//@
//@ \result!=null && (\forall JobRequest req; ; \result.contains(req) <==>
//@   (\old(getRequests()).contains(req) && req.matches(offer)) )
public Set<JobRequest> addOffer(JobOffer offer);
```

Domanda b)

Si specifichi in JML il metodo `getOffer`.

Soluzione

```
//@requires req != null
//@
//@ensures
//@ (\exists Offer o; \old(getOffers()).contains(o); req.matches(o)) ==> (
//@   unchangedRequests && getOffers().size()==\old(getOffers()).size()-1 &&
//@   (\exists int i; i>=0 && i<\old(getOffers()).size();
//@     req.matches(\old(getOffers()).get(i)) &&
//@     (\forall int j; j>=0 && j<i; !req.matches(\old(getOffers()).get(j)) ) &&
//@     (\forall int j; j>=0 && j<i; getOffers().get(j)==\old(getOffers()).get(j) ) &&
//@     (\forall int j; j>i && j<\old(getOffers()).size();
//@       getOffers().get(j-1)==\old(getOffers()).get(j) ) &&
//@     \result.equals(\old(getOffers())).get(i) ) ) &&
//@
//@ (!\exists Offer o; \old(getOffers()).contains(o); req.matches(o)) ==>
//@   (unchangedOffers && unchangedRequests && \result == null)
public JobOffer getOffer(JobRequest req);
```

Domanda c)

Si consideri un'implementazione di `JobsDB` che fa uso di un `Set` per memorizzare le richieste di lavoro e di una `List` per memorizzare le offerte di lavoro, come mostrato di seguito.

```
public class JobsDB {
  private Set<JobRequest> requests;
  private List<JobOffer> offers;
  ... }
```

Per tale implementazione, fornire l'invariante di rappresentazione (representation invariant, RI) e la funzione di astrazione (abstraction function, AF).

Soluzione

L'invariante di rappresentazione deve semplicemente indicare che le due collezioni non sono nulle e non contengono valori nulli.

```
private invariant
requests != null && offers != null && !requests.contains(null) && !offers.contains(null)
```

La funzione di astrazione deve indicare che i valori ritornati da `getRequests` e `getOffers` sono tutti e soli quelli contenuti nelle collezioni.

```
private invariant
getOffers().containsAll(offers) && offers.containsAll(getOffers()) &&
(\forall int i; i>=0 && i<offers.size(); getOffers().get(i) == offers.get(i)) &&
getOffers().size() == offers.size()
```

Domanda d)

Si consideri una classe `RecentJobsDB` che modifica il metodo `addOffer(JobOffer offer)` (override) facendo in modo che ritorni solo le richieste di lavoro aggiunte nell'ultima settimana. `RecentJobsDB` può essere definita come sottoclasse di `JobsDB` in accordo con il principio di sostituzione? Motivare la propria risposta.

Si consideri invece una classe `RecentJobsDB2` che aggiunge un metodo `addOffer(JobOffer offer, int days)` (overload) che si comporta come `addOffer(JobOffer offer)` ma ritorna solo le richieste di lavoro aggiunte negli ultimi `days` giorni. `RecentJobsDB2` può essere definita come sottoclasse di `JobsDB` in accordo con il principio di sostituzione? Motivare la propria risposta.

Soluzione

`RecentJobsDB` non può essere definita come sottoclasse di `JobsDB` in quanto il metodo `addOffer` ridefinito indebolisce la post condizione del metodo originale, ritornando meno risultati.

`RecentJobsDB2` può invece essere definita come sottoclasse di `JobsDB` in quanto si tratta di un'estensione pura (aggiunta di un metodo) che, modificando la classe come il metodo `addOffer` originale, non può violare alcuna proprietà pubblica.

Esercizio 2

Si consideri nuovamente un'implementazione di `JobsDB` che fa uso di un `Set` per memorizzare le richieste di lavoro e di una `List` per memorizzare le offerte di lavoro, come mostrato di seguito.

```
public class JobsDB {
  private Set<JobRequest> requests;
  private List<JobOffer> offers;
  ... }
```

Domanda a)

Si completi l'implementazione considerando solo i metodi `getRequests`, `getOffers`, `addRequest`, `addOffer` e garantendo thread-safety, ovvero facendo in modo che diversi thread possano invocare i metodi in parallelo senza creare problemi di concorrenza. **Si favoriscano soluzioni che massimizzano le possibilità di esecuzione parallela da parte dei thread che invocano i diversi metodi**.

Soluzione

```
public class JobsDB {
  ...
  public Set<JobRequest> getRequests() {
    synchronized(requests) { return new HashSet<>(requests); } }

  public List<JobOffer> getOffers() {
    synchronized(offers) { return new ArrayList<>(offers); } }

  public void addRequest(JobRequest req) throws DuplicateException {
    synchronized(requests) {
      if (requests.contains(req)) throws new DuplicateException();
      requests.add(request); } }

  public Set<JobRequest> addOffer(JobOffer offer) {
    synchronized(offers) { offers.add(offer); }
    synchronized(requests) {
      return requests.stream()
                     .filter(req -> req.matches(offer))
                     .collect(Collectors.toSet());
      }  }  }
}
```

Si osservi che abbiamo evitato di esporre il rep, restituendo ogni volta una copia.

Domanda b)

Si implementi ora anche il metodo `getOffer` facendo in modo che, invece di ritornare `null`, sospenda il chiamante nel caso in cui nessuna offerta risulta compatibile con la richiesta. Risulta necessario modificare anche le implementazioni di altri metodi? Se si, come?

Soluzione

Occorre anche aggiungere una `notifyAll` al metodo `addOffer`, come mostrato di seguito.

```
public class JobsDB {
  ...
  public JobOffer getOffer(JobRequest req) {
    synchronized(offers) {
      while (offers.stream().noneMatch(offer -> req.matches(offer))) {
        offers.wait();
      }
      JobOffer offer = offers.stream().findFirst(offer -> req.matches(offer)).get();
      offers.remove(offer);
      return offer;
    }
  }

  public Set<JobRequest> addOffer(JobOffer offer) {
```

```
    synchronized(offers) {
      offers.add(offer);
      offers.notifyAll();
    }
    synchronized(requests) {
      return requests.stream()
                     .filter(req -> req.matches(offer))
                     .collect(Collectors.toSet());
      }  }  }
}
```

Esercizio 3

Si consideri la seguente classe di cui viene qui omessa l'ovvia implementazione:

```
public class Student {
private ...;
   public Student(String name, int year, ...) {...}
   public String name() {...}
   public int yearOfBirth() {...}
   public String countryOfBirth() {...}
   public String personalCode(){...}
}
```

Sia data una `List<Student> s` che contiene una lista di studenti opportunamente inizializzata.

Rispondere alle domande seguenti usando **esclusivamente** i costrutti della *programmazione funzionale* (*senza quindi usare while, if.. else, forEach,* ecc.)

Domanda a)

Completare l'istruzione seguente per assegnare alla variabile `youngForeigners` i codici persona degli studenti non italiani nati dopo il 2000 presenti nella lista `s`.

```
List<String> youngForeigners =
```

Soluzione

```
List<String> youngForeigners = s.stream()
            .filter(student -> !student.countryOfBirth().equals("Italy"))
            .filter(student -> student.yearOfBirth() > 2000)
            .map(student -> student.personalCode())
            .collect(Collectors.toList());
```

Domanda b)

Memorizzare in una variabile `nameOfYoungestChinese` lo studente cinese piú giovane della lista `s`. Successivamente, *solo se presente*, stampare a schermo il suo nome (senza utilizzare il costrutto if.. else di Java).

Soluzione

```
final Optional<Student> nameOfYoungestChinese = s.stream()
        .filter(student -> student.countryOfBirth().equals("China"))
        .reduce((stud1, stud2) -> stud1.yearOfBirth() > stud2.yearOfBirth() ? stud1 : stud2)

nameOfYoungestChinese.ifPresent(student -> System.out.println(student.name()));
```

Esercizio 4

Si considerino le seguenti classi Java:

```
class Person {
  protected String name;
  public Person(String name) { this.name = name; }
  public void drive(Car c) {
    System.out.println(name+" driving "+c.model());
  }
}
class Pilot extends Person {
  public Pilot(String name) { super(name); }
  public void drive(RacingCar c) {
    System.out.println("Pilot "+name+" driving "+c.model());
  }
  public void drive(RacingCar c, Pilot coPilot) {
    System.out.println("Pilot "+name+" driving "+c.model()+" with "+coPilot.name);
  }
}
class RallyPilot extends Pilot {
  public RallyPilot(String name) { super(name); }
  public void drive(Car c) {
    System.out.println("Rally pilot "+name+" driving "+c.model());
  }
}

class Car {
  public String model() { return "a car"; }
}
class RacingCar extends Car {
  public String model() { return "a racing car"; }
}
```

e il seguente frammento di codice le cui righe sono state numerate per riferimento:

```
1    Person p = new Pilot("Andrea");
2    Pilot pp = new RallyPilot("Laura");
3    Car c[] = new Car[2];
4    c[0] = new Car();
5    c[1] = new RacingCar();
6    RacingCar rc = new RacingCar();
7
8    p.drive(c[0]);
9    p.drive(c[1]);
10   p.drive(rc);
11   p.drive(rc, pp);
12   pp.drive(c[0]);
13   pp.drive(c[1]);
14   pp.drive(rc);
15   pp.drive((RacingCar)c[0], pp);
16   pp.drive(c[1], pp);
17   pp.drive((RacingCar)c[1], pp);
18   pp.drive(rc, pp);
```

Domanda a)

Scrivere il numero di riga delle istruzioni (se ne esistono) che generano un errore in compilazione o a run-time, chiarendo in quale momento (compile-time vs. run-time) viene sollevato l'errore e spiegandone brevemente le ragioni.

Soluzione

La riga 11 non compila: `p` è staticamente di classe `Person` e tale classe non possiede il metodo invocato.

La riga 15 restituisce una `ClassCastException` a tempo di esecuzione, `c[0]` è dinamicamente di classe `Car`.

La riga 16 non compila: `c[1]` è staticamente di classe `Car` e il metodo `drive` con due parametri della classe `Pilot` vuole come primo parametro una `RacingCar`.

Domanda b)

Supponendo di eliminare le eventuali righe che generano errori indicate sopra, si scriva, per ogni riga rimanente, il valore stampato in uscita.

Soluzione

```
Andrea driving a car
Andrea driving a racing car
Andrea driving a racing car
Rally pilot Laura driving a car
Rally pilot Laura driving a racing car
Pilot Laura driving a racing car
Pilot Laura driving a racing car with Laura
Pilot Laura driving a racing car with Laura
```